

On Solving a Class of Linear Semi-Infinite Programs by the Trigonometric Moment

Y. Xu · J. Desai · X. Yan

Abstract In this paper, we propose a new easily implementable method for solving a class of semi-infinite programs, where an approximate linear semidefinite program is constructed for the concerned semi-infinite program based on the duality theory of semi-infinite programs and the theory of the trigonometric moments. We obtain a $\frac{\ln K}{K}$ -optimal solution for the semi-infinite program, where K is the truncation factor. Moreover, we present some numerical examples to show the efficiency of the new method.

Keywords Semi-Infinite Program · Trigonometric Moment · Semidefinite Program · FFT

Mathematics Subject Classification (2000) 90C34 · 42A70

1 Introduction

In this paper, we focus on designing a new method to solve semi-infinite programs (SIP). There are two main approaches that have adopted in the literature for solving semi-infinite programs, namely *integration* and *discretization* [5, 12]. In the integration-based approach, the semi-infinite constraint is replaced with the equivalent integration form. Usually, since a closed-form solution for the integral is too difficult to be obtained for generic functions, some numerical approximation-based methods are often prescribed to compute the integral [9].

An alternative set of algorithms have been developed based on *discretization methods*, which include simplex-like methods and cutting plane methods [3]. A necessary feature of such discretization methods is the computation of interpolation points. However, since the subprograms from solving the semi-infinite program are usually not convex programs, determining the proper interpolation points is a difficult undertaking in practice. Hence, from a computational viewpoint, integration and

Y. Xu (✉)

Department of Mathematics, Southeast University, 2 Sipailou, Nanjing, Jiangsu Province, 210096 P. R. China E-mail: yi.xu1983@hotmail.com

J. Desai

Manufacturing and Industrial Engineering Cluster, School of Mechanical and Aerospace Engineering, Nanyang Technological University, Singapore 639798. E-mail: jdesai@ntu.edu.sg

X. Yan

Department of Mathematics, Taiyuan Normal University, Taiyuan 030012, Shanxi Province, 306019, P.R. China, E-mail: xihong1@e.ntu.edu.sg

discretization-based approaches all have disadvantages. In addition to these two types of methods, other approaches, such as interior point methods [13], also suffer from similar drawbacks.

In [6, 7], Lasserre discussed a moment program for solving a polynomial program. We notice that the dual of a semi-infinite program is also a moment program. Hence, in this paper, we try to adopt a novel approach for solving a semi-infinite program based on the theory of the trigonometric moment, which could alleviate all of the aforementioned drawbacks.

The remainder of this paper is organized as follows. We present our motivation, contributions, and notations in Section 2, and then Section 3 introduces the trigonometric moment problem and an associated lemma that serves as a foundation to recast this problem as a semidefinite program. Then, in Section 4, a detailed step-wise analysis is provided to transform the trigonometric moment program into an approximate semidefinite program. Moreover, in Section 5, working under some mild assumptions, we will prove that the optimal solution of the semidefinite program got in Section 4 is just the approximate optimal solution of the original semi-infinite program. Some preliminary computational results on standard test problems taken from the literature are presented in Section 6, along with associated insights, and finally, Section 7 concludes this paper and presents some directions for future research.

2 Motivation, Contributions, and Notations

Consider the following linear *semi-infinite program* (**SIP**) and its dual program [5]:

$$(P) \quad \min \quad c^\top x \quad (1a)$$

$$s.t. \quad \sum_{j=1}^n a_j(t)x_j \leq a_0(t), \forall t \in [0, 2\pi], \quad (1b)$$

$$(D) \quad \max \quad - \int_0^{2\pi} a_0(t)\mu(dt) \quad (2a)$$

$$s.t. \quad \int_0^{2\pi} a_j(t)\mu(dt) = -c_j, j = 1, \dots, n, \quad (2b)$$

$$\mu \in \mathbf{P}([0, 2\pi]), \quad (2c)$$

where $a_j(t) \in \mathfrak{F}([0, 2\pi])$, $j = 0, \dots, n$, $\mathfrak{F}([0, 2\pi])$ denotes a set of functions defined on $[0, 2\pi]$, $\mathbf{P}([0, 2\pi])$ is a set of finite positive Borel measures on $[0, 2\pi]$. It is easy to see that Program (D) is essentially a *moment program*, and if the optimal measure μ of Program (D) is known, we can then successfully recover the optimal value of Program (P) from the solution of Program (D).

In [6, 7], Lasserre discussed the following moment program:

$$\min \quad \int a_0(t)\sigma(dt) \quad (3a)$$

$$s.t. \quad \sigma \in \mathbf{P}(T), \quad (3b)$$

where $T = \{t : a_j(t) \geq 0, j = 1, \dots, n\}$ is a semi-algebraic set and $\mathbf{P}(T)$ is the space of finite Borel measures on T . Lasserre [6, 7] concluded that the optimal value of Program (3) is equal to the optimal

value of

$$\min \quad a_0(t) \quad (4a)$$

$$s.t. \ a_j(t) \geq 0, \ j = 1, \dots, n, \quad (4b)$$

where $a_j(t)$, $j = 0, \dots, n$ are all polynomials. For solving the moment Program (3), Lasserre used the *theory of moments* and recasted Program (3) into a series of successive linear semidefinite programs.

Recognizing that Programs (D) and (3) are both moment programs, in this research, we also apply a moment-based approach for solving Program (D), leading to the solution of Program (P). However, our problem is intrinsically very different from the ones considered by Lasserre. First and foremost, Lasserre's method is based on the theory of the *Hamburger moment problem*, wherein $\sigma \in \mathbf{P}(\mathbb{R}^n)$, while the corresponding moment μ in Program (D) only belongs to $\mathbf{P}([0, 2\pi])$, which cannot be cast as a Hamburger moment problem. Second, as $a_j(t)$ can be any generic function and not necessarily a polynomial, we cannot apply Lasserre's method directly in this case. While we could certainly use a polynomial function $b_j(t)$ to approximate $a_j(t)$, possibly by using Taylor series expansions; or the least square method; or Newton's interpolation method, such an approach inherently has several disadvantages. To note a few, Taylor series expansions require a high order of differentiability for the involved functions $a_j(t)$; the underlying integrations that exist in a least squares method are computationally very expensive; and Newton's interpolation method fails to retain and exploit the nice linear structure of Program (P).

Hence, in this paper, we adopt a novel approach for solving Program (D) based on the theory of the trigonometric moment that not only alleviates all of the aforementioned drawbacks, but also provides a very viable approach for solving large-scale SIP. We transform the trigonometric moment program into a linear semidefinite program. Moreover, we prove that the optimal solution of this linear semidefinite program is a $\frac{\ln K}{K}$ -optimal solution of the original semi-infinite program, where K is the truncation factor.

In closing this section, we introduce some basic notations and related definitions that will be used in our subsequent analysis.

Notations and Symbols:

Let $\mathbb{R}^{n \times n}$ and $\mathbb{C}^{n \times n}$ denote the set of all $n \times n$ real and complex matrices, respectively, and let $\mathbb{RS}^n$ and $\mathbb{CS}^n$ represent the set of all $n \times n$ real symmetric and complex conjugate matrices, respectively. We write $A \succeq 0$ if A is positive semidefinite. Let $\mathbb{CS}_+^n = \{A \in \mathbb{CS}^n : A \succeq 0\}$ and $\mathbb{RS}_+^n = \{A \in \mathbb{RS}^n : A \succeq 0\}$ denote the set of complex conjugate and real symmetric positive semidefinite matrices, respectively. We use $\mathcal{C}$ to denote a matrix set $\{\mathcal{C}_j\}$, $\mathcal{C}_j \in \mathbb{R}^{n \times n}$, and denote $\mathcal{C}X$ as a vector, whose element $(\mathcal{C}X)_i = \text{Tr}(\mathcal{C}_i X)$, where $\text{Tr}(A) = \sum A_{j,j}$; $\mathcal{C}^T x$ is defined as $\sum \mathcal{C}_j x_j$.

We set i to represent the *imaginary unit*, such that $i^2 = -1$; ξ^T means the conjugate transpose of ξ ; $\bar{\alpha}$ denotes the conjugate of complex number α ; $\|\cdot\|$ is 2-norm; $B(x^*, \rho)$ is a hyperball of radius ρ centered at x^* , i.e., $\{x \in \mathbb{R}^n : \|x - x^*\| \leq \rho\}$; $\|\Omega\| = \max_{x \in \Omega} \|x\|$; $d(x^*, \Omega) = \min\{\|x - x^*\| : x \in \Omega\}$; $\mathbb{P}_\Omega(x^*) = \arg \min\{\|x - x^*\| : x \in \Omega\}$; $\text{int}(\Omega)$ is the interior set of Ω ; $l(a, b)$ represents the line connecting points a and b ; $S(P)$, $F(P)$, and $v(P)$ are the optimal solution set, the feasible set, and the optimal value of Program (P), respectively.

3 The Trigonometric Moment Problem

In a trigonometric moment problem [1, 2], given a finite sequence $\{\alpha_0, \dots, \alpha_n\}$, we are interested in determining whether there exists a positive Borel measure μ , defined on the interval $[0, 2\pi]$, such that

$$\alpha_k = \frac{1}{2\pi} \int_0^{2\pi} e^{-ikt} \mu(dt), \quad k = 0, \dots, n.$$

In other words, an affirmative answer to this moment problem would imply that $\{\alpha_0, \dots, \alpha_n\}$ are the first $n + 1$ Fourier coefficients of some positive Borel measure μ on $[0, 2\pi]$. In fact, it has been demonstrated in [1, 2] that the trigonometric moment problem does indeed have a solution if the condition in the following lemma holds true.

Lemma 3.1 The trigonometric moment problem has a solution, i.e., $\{\alpha_k\}_{k=0}^n$ is a valid sequence of Fourier coefficients of some positive Borel measure μ defined on $[0, 2\pi]$, if and only if there exists a Toeplitz matrix $A \in \mathbb{C}^{n+1}$ that is positive semidefinite, that is

$$A = \begin{pmatrix} \alpha_0 & \alpha_1 & \cdots & \alpha_n \\ \bar{\alpha}_1 & \alpha_0 & \cdots & \alpha_{n-1} \\ \vdots & \vdots & \ddots & \vdots \\ \bar{\alpha}_n & \bar{\alpha}_{n-1} & \cdots & \alpha_0 \end{pmatrix} \succeq 0.$$

Remark 3.1 (a) Note that the existence of a solution for the trigonometric moment problem is similar to the one of the Hamburger moment problem, which also requires matrix A to be positive semidefinite. However, the matrix A in the Hamburger moment problem is a Hankel matrix, and not a Toeplitz matrix, as needed for the problem under consideration.

(b) Matrix A in Lemma 3.1 is not only a Toeplitz matrix, but it is also a conjugate matrix. Hence, when $A \succeq 0$, α_0 must be a real and nonnegative number.

4 Transforming a Linear Semi-Infinite Program into an Approximate Semidefinite Program

The foregoing Dirichlet conditions of $f(t)$ on $[0, 2\pi]$:

- (1) $f(t)$ is absolutely integrable over $[0, 2\pi]$;
- (2) $f(t)$ has a finite number of extrema in $[0, 2\pi]$;
- (3) $f(t)$ has a finite number of discontinuities in $[0, 2\pi]$.

are well-known sufficiency conditions for the existence of a Fourier transformation of $f(t)$. Working under Dirichlet conditions of $a_j(t)$, $j = 0, \dots, n$ on $[0, 2\pi]$, $a_j(t)$ can be represented via its Fourier transform as

$$\hat{a}_j(t) = \frac{1}{2\pi} \sum_{k=-\infty}^{+\infty} a_{j,k} e^{-ikt},$$

where

$$a_{j,k} = \frac{1}{2\pi} \int_0^{2\pi} a_j(t) e^{ikt} dt. \quad (5)$$

Substituting $\hat{a}_j(t)$ in lieu of $a_j(t)$ into Program (D), the resulting *Fourier Dual* Program (FD) can be written as:

$$(FD) \max \quad -\frac{1}{2\pi} \int_0^{2\pi} \sum_{k=-\infty}^{+\infty} a_{0,k} e^{-ikt} \mu(dt) \quad (6a)$$

$$s.t. \quad \frac{1}{2\pi} \int_0^{2\pi} \sum_{k=-\infty}^{+\infty} a_{j,k} e^{-ikt} \mu(dt) = -c_j, \quad j = 1, \dots, n, \quad (6b)$$

$$\mu \in \mathbf{P}([0, 2\pi]). \quad (6c)$$

As Program (FD) cannot be directly solved due to the presence of the infinite summation terms present in this program, we consider a truncated version of this program, denoted as Program (FD_K), given by:

$$(FD_K) \max \quad -\frac{1}{2\pi} \int_0^{2\pi} \sum_{k=-K}^K a_{0,k} e^{-ikt} \mu(dt) \quad (7a)$$

$$s.t. \quad \frac{1}{2\pi} \int_0^{2\pi} \sum_{k=-K}^K a_{j,k} e^{-ikt} \mu(dt) = -c_j, \quad j = 1, \dots, n, \quad (7b)$$

$$\mu \in \mathbf{P}([0, 2\pi]), \quad (7c)$$

where K , a positive integer, is the truncation factor.

Interchanging the summation and integral terms, this program can in turn be represented as:

$$(FD_K) \max \quad - \sum_{k=-K}^K a_{0,k} \frac{1}{2\pi} \int_0^{2\pi} e^{-ikt} \mu(dt) \quad (8a)$$

$$s.t. \quad \sum_{k=-K}^K a_{j,k} \frac{1}{2\pi} \int_0^{2\pi} e^{-ikt} \mu(dt) = -c_j, \quad j = 1, \dots, n, \quad (8b)$$

$$\mu \in \mathbf{P}([0, 2\pi]). \quad (8c)$$

Now, let $y_k = \frac{1}{2\pi} \int_0^{2\pi} e^{-ikt} \mu(dt)$, $k = -K, \dots, K$. It is easy to see that y_0 is a real number, in general, y_k satisfies $y_{-k} = \bar{y}_k$.

From Lemma 3.1, it follows that the matrix composed of y_k satisfies

$$\begin{pmatrix} y_0 & y_1 & \cdots & y_K \\ \bar{y}_1 & y_0 & \cdots & y_{K-1} \\ \vdots & \vdots & \ddots & \vdots \\ \bar{y}_K & \bar{y}_{K-1} & \cdots & y_0 \end{pmatrix} = \begin{pmatrix} y_0 & y_1 & \cdots & y_K \\ y_{-1} & y_0 & \cdots & y_{K-1} \\ \vdots & \vdots & \ddots & \vdots \\ y_{-K} & y_{-K+1} & \cdots & y_0 \end{pmatrix} \succeq 0.$$

Conversely, once again by Lemma 3.1, if y_k satisfies the above positive semidefiniteness condition, then, there must be a positive Borel measure μ , defined on the interval $[0, 2\pi]$, such that

$$y_k = \frac{1}{2\pi} \int_0^{2\pi} e^{-ikt} \mu(dt), \quad k = -K, \dots, K.$$

Incorporating the results of Lemma 3.1 into Program (FD_K) , we get the following *Trigonometric Fourier Dual Program* (TFD_K) variant of Program (P) :

$$(TFD_K) \max \quad - \sum_{k=-K}^K a_{0,k} y_k \quad (9a)$$

$$s.t. \quad \sum_{k=-K}^K a_{j,k} y_k = -c_j, j = 1, \dots, n, \quad (9b)$$

$$\begin{pmatrix} y_0 & y_1 & \cdots & y_K \\ y_{-1} & y_0 & \cdots & y_{K-1} \\ \vdots & \vdots & \ddots & \vdots \\ y_{-K} & y_{-K+1} & \cdots & y_0 \end{pmatrix} \succeq 0. \quad (9c)$$

It is clear that Program (TFD_K) is not a conventional optimization problem as both $a_{j,k}$ and y_k in Program (TFD_K) may be complex numbers, for all $k \neq 0$. Hence, for solving Program (TFD_K) , we adopt a novel approach by separating each of the program parameters and variables into their respective real and imaginary components, and then using the properties of the involved matrices to derive an equivalent program defined purely in terms of real-valued variables. Towards this end, set

$$y_k = w_k + i v_k, \quad (10a)$$

$$a_{j,k} = r_{j,k} + i s_{j,k}, \quad (10b)$$

where $w_k, v_k, r_{j,k}$, and $s_{j,k}$ are all real numbers.

Then,

$$w_k = w_{-k}, v_k = -v_{-k}, v_0 = 0 \quad (11a)$$

$$r_{j,k} = r_{j,-k}, s_{j,k} = -s_{j,-k}, \quad (11b)$$

$$s_{j,0} = 0, j = 0, \dots, n, \quad (11c)$$

where equations (11b) and (11c) are a byproduct of the definition of the Fourier transformation of $a_j(t)$.

Substituting these resulting terms into Program (TFD_K) , we get

$$\sum_{k=-K}^K a_{j,k} y_k = \sum_{k=-K}^K r_{j,k} w_k - s_{j,k} v_k, \quad j = 0, \dots, n,$$

and

$$\begin{pmatrix} y_0 & y_1 & \cdots & y_K \\ y_{-1} & y_0 & \cdots & y_{K-1} \\ \vdots & \vdots & \ddots & \vdots \\ y_{-K} & y_{-K+1} & \cdots & y_0 \end{pmatrix} = \begin{pmatrix} w_0 & w_1 & \cdots & w_K \\ w_1 & w_0 & \cdots & w_{K-1} \\ \vdots & \vdots & \ddots & \vdots \\ w_K & w_{K-1} & \cdots & w_0 \end{pmatrix} + i \begin{pmatrix} 0 & v_1 & \cdots & v_K \\ -v_1 & 0 & \cdots & v_{K-1} \\ \vdots & \vdots & \ddots & \vdots \\ -v_K & -v_{K-1} & \cdots & 0 \end{pmatrix} \succeq 0.$$

We are now ready to present Lemma 4.1, which accords us with a mechanism to transform $Y \in \mathbb{CS}^K, Y \succeq 0$ into a real-valued positive semidefinite matrix.

Lemma 4.1

$$\begin{pmatrix} y_0 & y_1 & \cdots & y_K \\ y_{-1} & y_0 & \cdots & y_{K-1} \\ \vdots & \vdots & \ddots & \vdots \\ y_{-K} & y_{1-K} & \cdots & y_0 \end{pmatrix} \succeq 0 \Leftrightarrow \begin{pmatrix} w_0 & w_1 & \cdots & w_K & 0 & -v_1 & \cdots & -v_K \\ w_1 & w_0 & \cdots & w_{K-1} & v_1 & 0 & \cdots & -v_{K-1} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ w_K & w_{K-1} & \cdots & w_0 & v_K & v_{K-1} & \cdots & 0 \\ 0 & v_1 & \cdots & v_K & w_0 & w_1 & \cdots & w_K \\ -v_1 & 0 & \cdots & v_{K-1} & w_1 & w_0 & \cdots & w_{K-1} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ -v_K & -v_{K-1} & \cdots & 0 & w_K & w_{K-1} & \cdots & w_0 \end{pmatrix} \succeq 0.$$

Proof Denote

$$Y = \begin{pmatrix} y_0 & y_1 & \cdots & y_K \\ y_{-1} & y_0 & \cdots & y_{K-1} \\ \vdots & \vdots & \ddots & \vdots \\ y_{-K} & y_{1-K} & \cdots & y_0 \end{pmatrix}, W = \begin{pmatrix} w_0 & w_1 & \cdots & w_K \\ w_1 & w_0 & \cdots & w_{K-1} \\ \vdots & \vdots & \ddots & \vdots \\ w_K & w_{K-1} & \cdots & w_0 \end{pmatrix}, V = \begin{pmatrix} 0 & v_1 & \cdots & v_K \\ -v_1 & 0 & \cdots & v_{K-1} \\ \vdots & \vdots & \ddots & \vdots \\ -v_K & -v_{K-1} & \cdots & 0 \end{pmatrix},$$

where, clearly $W = W^\top, V = -V^\top$, with $W, V \in \mathbb{R}^{(K+1) \times (K+1)}$.

As $Y \succeq 0$, this implies that $(p + iq)^T Y (p + iq) \geq 0, \forall p, q \in \mathbb{R}^{(K+1) \times 1}$. Expanding this product, we get

$$(p + iq)^T Y (p + iq) = p^\top W p - p^\top V q + q^\top W q + q^\top V p + i(p^\top W q + p^\top V p - q^\top W p + q^\top V q).$$

Utilizing the property that W is a symmetric matrix and V is a skew-symmetric matrix, which implies $p^\top W q = q^\top W p$ and $p^\top V p = q^\top V q = 0$, we get that $Y \succeq 0$ if and only if

$$p^\top W p - p^\top V q + q^\top W q + q^\top V p \geq 0, \forall p, q \in \mathbb{R}^{(K+1) \times 1},$$

which in turn can be compactly expressed as a real, symmetric matrix

$$\begin{pmatrix} W & V^\top \\ V & W \end{pmatrix} \succeq 0,$$

and this completes the proof. $\square$

Using the variable transformations (10a) and (10b), in concert with Lemma 3.1, the following equivalent form of Program (TFD_K) , now defined in terms of only real-valued variables, can be obtained as follows:

$$(TFD_K) \quad \max \quad - \sum_{k=-K}^K r_{0,k} w_k - s_{0,k} v_k \quad (12a)$$

$$s.t. \quad \sum_{k=-K}^K r_{j,k} w_k - s_{j,k} v_k = -c_j, \quad j = 1, \dots, n, \quad (12b)$$

$$v_k = -v_{-k}, w_k = w_{-k}, \quad k = 0, \dots, K, \quad (12c)$$

$$\begin{pmatrix} w_0 & w_1 & \cdots & w_K & 0 & -v_1 & \cdots & -v_K \\ w_1 & w_0 & \cdots & w_{K-1} & v_1 & 0 & \cdots & -v_{K-1} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ w_K & w_{K-1} & \cdots & w_0 & v_K & v_{K-1} & \cdots & 0 \\ 0 & v_1 & \cdots & v_K & w_0 & w_1 & \cdots & w_K \\ -v_1 & 0 & \cdots & v_{K-1} & w_1 & w_0 & \cdots & w_{K-1} \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ -v_K & -v_{K-1} & \cdots & 0 & w_K & w_{K-1} & \cdots & w_0 \end{pmatrix} \succeq 0. \quad (12d)$$

A straightforward extension of the above discussion results in the following theorem, it is obviously that $v(TFD_K) = v(FD_K)$.

Let

$$\begin{aligned} \bar{C}_1^K &= \{\bar{C}_{1,j}^K\}_{j=0}^K, \bar{C}_{1,j}^K = \begin{pmatrix} C_{1,j}^K & 0 \\ 0 & C_{1,j}^K \end{pmatrix}, (C_{1,j}^K)_{k,l} = \begin{cases} 1, & |k-l| = j \\ 0, & \text{else} \end{cases}, C_{1,j}^K \in \mathbb{R}^{(K+1) \times (K+1)}, \\ \bar{C}_2^K &= \{\bar{C}_{2,j}^K\}_{j=1}^K, \bar{C}_{2,j}^K = \begin{pmatrix} 0 & -C_{2,j}^K \\ C_{2,j}^K & 0 \end{pmatrix}, (C_{2,j}^K)_{k,l} = \begin{cases} 1, & l-k = j \\ -1, & k-j = j \\ 0, & \text{else} \end{cases}, C_{2,j}^K \in \mathbb{R}^{(K+1) \times (K+1)}, \\ p_j^K &= \begin{cases} r_{0,0} & j=0 \\ r_{0,j} + r_{0,-j} & \text{else} \end{cases}, p^K \in \mathbb{R}^{(K+1) \times 1}, q_j^K = s_{0,j} - s_{0,-j}, q^K \in \mathbb{R}^{K \times 1}, \\ E_{j,k}^K &= \begin{cases} r_{j,0}, & k=0 \\ r_{j,k} + r_{j,-k}, & \text{else} \end{cases}, E^K \in \mathbb{R}^{n \times (K+1)}, F_{j,k}^K = s_{j,k} - s_{j,-k}, F^K \in \mathbb{R}^{n \times K}, \\ w &= (w_0, w_1, \dots, w_K)^T, v = (v_1, \dots, v_K)^T, \end{aligned}$$

(TFD_K) could be rewritten as

$$\begin{aligned} (TFD_K) \quad & \max \quad (p^K)^T w - (q^K)^T v \\ \text{s.t.} \quad & E^K w - F^K v = -c, \\ & (\bar{C}_1^K)^T w + (\bar{C}_2^K)^T v \succeq 0. \end{aligned}$$

Furthermore, its dual program (TFP_K) is the following semidefinite program:

$$\begin{aligned} (TFP_K) \quad & \min \quad c^T x \\ \text{s.t.} \quad & \bar{C}_1^K Y + (E^K)^T x + p^K = 0, \\ & \bar{C}_2^K Y - (F^K)^T x - q^K = 0, \\ & Y \succeq 0. \end{aligned}$$

5 Error Analysis and Algorithm

Firstly, we introduce the definition of ε -optimal solution here.

Definition 5.1 We call a point x^* is a ε -optimal solution of (P) , if there is a constant $L > 0$ such that

$$(a) \quad |c^T x^* - v(P)| \leq L\varepsilon;$$

(b) $d(x^*, F(P)) \leq L\varepsilon$.

Lemma 5.1 If $\exists \tilde{x} \in \mathbb{R}^n$ and $Q < 0$ such that

$$\sum_{j=1}^n a_j(t) \tilde{x}_j \leq Q, \forall t \in [0, 2\pi],$$

then, there exists $\sigma > 0$, only depended on $\tilde{x}$ and Q , such that

$$d(x, F(P)) \leq \sigma \max\left\{ \max_{t \in [0, 2\pi]} \sum_{j=1}^n a_j(t) x_j - a_0(t), 0 \right\}$$

Proof If $x \in F(P)$, it is true, so we only need to consider $x \notin F(P)$. Let $y = \mathbb{P}_{F(P)}(x)$, there must be $\lambda_j \geq 0, t_j \in [0, 2\pi], j = 1, \dots, l$ such that $0 \neq x - y = A^T \lambda$, $Ay = b$ where $A \in \mathbb{R}^{l \times n}, A_{k,j} = a_j(t_k), b = (a_0(t_1), a_0(t_2), \dots, a_0(t_l))^T$, so $d(x, F(P)) = \|x - y\|$.

Then, $\max\left\{ \max_{t \in [0, 2\pi]} \sum_{j=1}^n a_j(t) x_j - a_0(t), 0 \right\} = \max_{j=1, \dots, l} \{ \max\{ (AA^T \lambda)_j, 0 \} \}$. Since

$$\sqrt{n} \max_{j=1, \dots, l} \{ \max\{ (AA^T \lambda)_j, 0 \} \} \|\lambda\| \geq \sum_{j=1, \dots, l} \lambda_j \max\{ (AA^T \lambda)_j, 0 \} \geq \sum_{j=1, \dots, l} \lambda_j (AA^T \lambda)_j = \|A^T \lambda\|^2,$$

then,

$$\frac{\sqrt{n} \max_{j=1, \dots, l} \{ \max\{ (AA^T \lambda)_j, 0 \} \}}{\|A^T \lambda\|} \geq \frac{\|A^T \lambda\|}{\|\lambda\|}.$$

Since

$$\sum_{j=1}^n a_j(t) \tilde{x}_j \leq Q, \forall t \in [0, 2\pi],$$

we have $A(-\tilde{x}) > 0$, so

$$\|\tilde{x}\| \|A^T \lambda\| \geq - \sum_{j=1}^n \tilde{x}_j (A^T \lambda)_j \geq - \max_{j=1, \dots, n} \{ (A\tilde{x})_j \} \|\lambda\| \geq -Q \|\lambda\|,$$

then,

$$\begin{aligned} d(x, F(P)) &= \|x - y\| = \|A^T \lambda\| \leq \frac{\|\lambda\|}{\|A^T \lambda\|} \sqrt{n} \max_{j=1, \dots, l} \{ \max\{ (AA^T \lambda)_j, 0 \} \} \\ &\leq \frac{\sqrt{n} \|\tilde{x}\|}{-Q} \max_{j=1, \dots, l} \{ \max\{ (AA^T \lambda)_j, 0 \} \} \\ &= \sigma \max\left\{ \max_{t \in [0, 2\pi]} \sum_{j=1}^n a_j(t) x_j - a_0(t), 0 \right\}, \end{aligned}$$

where $\sigma = \frac{\sqrt{n} \|\tilde{x}\|}{-Q}$. □

Now, we are ready to present our main result.

Theorem 5.1 Supposing the following conditions hold:

(a) $a_j(t), j = 0, \dots, n$ are Lipschitz continuous functions on $[0, 2\pi]$;

- (b) $a_j(0) = a_j(2\pi), j = 0, \dots, n$;
(c) The Slater condition holds for Program (P): there exists a point $\hat{x}$ satisfied

$$\sum_{j=1}^n a_j(t) \hat{x}_j < a_0(t), \forall t \in [0, 2\pi];$$

- (d) $\exists \tilde{x} \in \mathbb{R}^n$ and $Q < 0$ such that

$$\sum_{j=1}^n a_j(t) \tilde{x}_j \leq Q, \forall t \in [0, 2\pi];$$

- (e) $c \neq 0, S(P) \neq \emptyset, \|S(P)\| \leq R$,

then,

- (a) the Slater condition holds for Program (FP_K) , when K is large enough, where

$$\begin{aligned} (FP_K) \quad & \min \quad c^\top x \\ & s.t. \quad \sum_{j=1}^n \hat{a}_j^K(t) x_j \leq \hat{a}_0^K(t), \forall t \in [0, 2\pi] \end{aligned}$$

is the dual program of Program (FD_K) ;

- (b) if $\tilde{x}^K \in S(FP_K)$, $\tilde{x}^K$ is a $\frac{\ln K}{K}$ -optimal solution of Program (P), where

$$\begin{aligned} (AFP_K) \quad & \min \quad c^\top x \\ & s.t. \quad \sum_{j=1}^n \hat{a}_j^K(t) x_j \leq \hat{a}_0^K(t) + 2(nR + 1)A \frac{\ln K}{K}, \forall t \in [0, 2\pi], \end{aligned}$$

when K is large enough;

- (c) if $(\tilde{x}^*)^K \in S(FP_K)$, $(\tilde{x}^*)^K$ is a $\frac{\ln K}{K}$ -optimal solution of Program (P), when K is large enough;
(d) $(x^*)^K$ is a $\frac{\ln K}{K}$ -optimal solution of Program (P), when K is large enough, where $((x^*)^K, (Y^*)^K) \in S(TFP_K)$ with some $(Y^*)^K \succeq 0$.

Proof Since $a_j(t)$ is a Lipschitz function on $[0, 2\pi]$ and $a_j(0) = a_j(2\pi)$, we always could extend $a_j(t)$ to $\check{a}_j(t)$ defined on $\mathbb{R}$ such that $\check{a}_j(t)$ is a Lipschitz function on $\mathbb{R}$. From Corollary I in page 22 of [8], it follows that there exists a $A > 0$ such that

$$|a_j(t) - \hat{a}_j^K(t)| \leq A \frac{\ln K}{K}, \forall t \in [0, 2\pi],$$

where $\hat{a}_j^K(t)$ is the Fourier transformation of $a_j(t)$.

- (a) Let K be large enough such that

$$\min_{t \in [0, 2\pi]} (a_0(t) - \sum_{j=1}^n a_j(t) \hat{x}_j) - (n\|\hat{x}\| + 1)A \frac{\ln K}{K} > 0,$$

and

$$\tilde{\rho} = 0.9 \frac{\min_{t \in [0, 2\pi]} (a_0(t) - \sum_{j=1}^n a_j(t) \hat{x}_j) - (n \|\hat{x}\| + 1) A \frac{\ln K}{K}}{\max_{t \in [0, 2\pi]} \sum_{j=1}^n |a_j(t)| + n A \frac{\ln K}{K}} > 0,$$

then, $\forall \tilde{x} \in B(\hat{x}, \tilde{\rho})$,

$$\begin{aligned} & \max_{t \in [0, 2\pi]} \sum_{j=1}^n \hat{a}_j^K(t) \tilde{x}_j - a_0^K(t) \\ &= \max_{t \in [0, 2\pi]} \sum_{j=1}^n \hat{a}_j^K(t) \hat{x}_j - a_0^K(t) + \sum_{j=1}^n \hat{a}_j^K(t) (\tilde{x}_j - \hat{x}_j) \\ &= \max_{t \in [0, 2\pi]} \sum_{j=1}^n (\hat{a}_j^K(t) - a_j(t)) \hat{x}_j - (a_0^K(t) - a_0(t)) + \sum_{j=1}^n (\hat{a}_j^K(t) - a_j(t)) (\tilde{x}_j - \hat{x}_j) \\ &\quad + \sum_{j=1}^n a_j(t) \hat{x}_j - a_0(t) + \sum_{j=1}^n a_j(t) (\tilde{x}_j - \hat{x}_j) \\ &\leq (n \|\hat{x}\| + 1) A \frac{\ln K}{K} + \tilde{\rho} n A \frac{\ln K}{K} + \tilde{\rho} \max_{t \in [0, 2\pi]} \sum_{j=1}^n |a_j(t)| + \max_{t \in [0, 2\pi]} \sum_{j=1}^n a_j(t) \hat{x}_j - a_0(t) \\ &< 0, \end{aligned}$$

so the slater condition holds for Program (FP_K) , when K is large enough.

(b) It is obviously that the slater condition also holds for Program (AFP_K) . Furthermore, since $S(P) \neq \emptyset$, $\|S(P)\| \leq R$, then, $\forall \bar{x} \in S(P)$,

$$\begin{aligned} & \max_{t \in [0, 2\pi]} \sum_{j=1}^n \hat{a}_j^K(t) \bar{x}_j - \hat{a}_0^K(t) \\ &= \max_{t \in [0, 2\pi]} \sum_{j=1}^n (\hat{a}_j^K(t) - a_j(t)) \bar{x}_j - (\hat{a}_0^K(t) - a_0(t)) + \sum_{j=1}^n a_j(t) \bar{x}_j - a_0(t) \\ &< 2(nR + 1) A \frac{\ln K}{K}, \end{aligned}$$

which means $\bar{x} \in \text{int}(F(AFP_K))$ and $v(P) > v(AFP_K)$.

If $\|S(AFP_K)\| > 2R$, there exist $\{K_i\}$ such that $\|\tilde{x}^{K_i}\| > 2R$, $\tilde{x}^{K_i} \in S(AFP_{K_i})$. Taken $\tilde{\tilde{x}}^{K_i} \in l(\bar{x}, \tilde{x}^{K_i})$ satisfied $\|\tilde{\tilde{x}}^{K_i}\| = 2R$, where $\bar{x} \in S(P)$, we have $v(AFP_{K_i}) \leq c^T \tilde{\tilde{x}}^{K_i} < v(P)$, $\tilde{\tilde{x}}^{K_i} \in F(AFP_{K_i})$. By Lemma 5.1,

$$d(\tilde{\tilde{x}}^{K_i}, F(P)) \leq \sigma \max\left\{ \max_{t \in [0, 2\pi]} \sum_{j=1}^n a_j(t) \tilde{\tilde{x}}_j^{K_i} - a_0(t), 0 \right\}.$$

Noticing that

$$\max_{t \in [0, 2\pi]} \sum_{j=1}^n a_j(t) \tilde{\tilde{x}}_j^{K_i} - a_0(t)$$

$$\begin{aligned}
&= \max_{t \in [0, 2\pi]} \sum_{j=1}^n (a_j(t) - \hat{a}_j^{K_i}(t)) \tilde{x}_j^{K_i} - (a_0(t) - \hat{a}_0^{K_i}(t)) + \sum_{j=1}^n \hat{a}_j^{K_i}(t) \tilde{x}_j^{K_i} - \hat{a}_0^{K_i}(t) \\
&\leq (2nR + 1)A \frac{\ln K_i}{K_i} + 2(nR + 1)A \frac{\ln K_i}{K_i} \\
&= (4nR + 3)A \frac{\ln K_i}{K_i}.
\end{aligned}$$

Then,

$$d(\tilde{x}^{K_i}, F(P)) \leq \sigma(4nR + 3)A \frac{\ln K_i}{K_i}.$$

When $K_i \rightarrow \infty$, $d(\tilde{x}^{K_i}, F(P)) \rightarrow 0$. For any accumulation point $\tilde{x}$ of $\tilde{x}^{K_i}$, we have $c^T \tilde{x} = v(P)$, $\|\tilde{x}\| = 2R$, which is against to $\|S(P)\| \leq R$. So $\|S(AFP_K)\| \leq 2R$, when K is large enough.

By the same way, we could prove that

$$d(\tilde{x}^K, F(P)) \leq \sigma(4nR + 3)A \frac{\ln K}{K},$$

where $\tilde{x}^K \in S(AFP_K)$. Then,

$$|c^T \tilde{x}^K - c^T \mathbb{P}_{F(P)} \tilde{x}^K| \leq \|c\| \sigma(4nR + 3)A \frac{\ln K}{K},$$

which means

$$-L \frac{\ln K}{K} \leq v(AFP_K) - v(P) \leq 0,$$

where $L = \|c\| \sigma(4nR + 3)A$.

(c) We will prove there exists two constants $L > 0$ and $U > 0$ such that

$$-L \frac{\ln K}{K} \leq v(FP_K) - v(P) \leq U \frac{\ln K}{K}.$$

Since the Slater condition holds for Program (AFP_K) ,

$$\begin{aligned}
0 &\geq v(AFP_K) - v(FD_K) = v(AFD_K) - v(FD_K) \\
&\quad - \max 2(nR + 1)A \frac{\ln K}{K} \int_0^{2\pi} \mu(dt) \\
&\geq \quad s.t. \quad \int_0^{2\pi} \hat{a}_j^K(t) \mu(dt) = -c_j, \quad j = 1, \dots, n, \\
&\quad \mu \in \mathbf{P}([0, 2\pi]),
\end{aligned}$$

where

$$\begin{aligned}
(AFD_K) \quad &\max - \int_0^{2\pi} \hat{a}_0^K(t) + 2(nR + 1)A \frac{\ln K}{K} \mu(dt) \\
&\quad s.t. \quad \int_0^{2\pi} \hat{a}_j^K(t) \mu(dt) = -c_j, \quad j = 1, \dots, n, \\
&\quad \mu \in \mathbf{P}([0, 2\pi])
\end{aligned}$$

is the dual program of Program (AFP_K) .

We consider Program (TD) and its dual Program (TP) :

$$(TD) \quad \max \quad -\frac{Q}{2} \int_0^{2\pi} \mu(dt)$$

$$s.t. \quad \int_0^{2\pi} \hat{a}_j^K(t) \mu(dt) = -c_j, j = 1, \dots, n,$$

$$\mu \in \mathbf{P}([0, 2\pi]),$$

$$(TP) \quad \min \quad c^T x$$

$$s.t. \quad \sum_{j=1}^n \hat{a}_j^K(t) x_j \leq \frac{Q}{2}, \forall t \in [0, 2\pi].$$

It is easy to see that $v(TP) \geq v(TD) \geq 0$, since $F(TD) = F(AFD_K)$ is not empty. Let K be large enough such that $A \frac{\ln K}{K} \leq \frac{-Q}{2n\|\tilde{x}\|}$, then,

$$\sum_{j=1}^n \hat{a}_j^K(t) \tilde{x}_j = \sum_{j=1}^n (\hat{a}_j^K(t) - a_j(t)) \tilde{x}_j + \sum_{j=1}^n a_j(t) \tilde{x}_j \leq n\|\tilde{x}\| A \frac{\ln K}{K} + Q \leq \frac{Q}{2},$$

which means that $\tilde{x}$ is a feasible point of Program (TP) , then,

$$0 \leq v(TD) \leq v(TP) \leq c^T \tilde{x}.$$

Let $U = \frac{-4(nR+1)c^T \tilde{x}}{Q} A$, we have that

$$-U \frac{\ln K}{K} \leq v(AFD_K) - v(FD_K) = v(AFP_K) - v(FD_K) \leq 0,$$

since the slater condition holds for Program (AFP_K) , when K is large enough. So when K is large enough, we have

$$-L \frac{\ln K}{K} \leq v(FD_K) - v(P) = v(FP_K) - v(P) \leq U \frac{\ln K}{K}.$$

Furthermore, we will prove

$$d((x^*)^K, F(P)) \leq \sigma(4nR+3)A \frac{\ln K}{K},$$

where $(x^*)^K \in S(FP_K)$, when K is large enough. If $\|S(FP_K)\| > 2R$, there exists $\{K_i\}$ such that $(x^*)^{K_i} \in S(FP_{K_i})$, $\|(x^*)^{K_i}\| > 2R$. Taken $(x^{**})^{K_i} \in l(\bar{x}, (x^*)^{K_i})$ satisfied $\|(x^{**})^{K_i}\| = 2R$, where $\bar{x} \in S(P)$, then,

$$-L \frac{\ln K_i}{K_i} \leq c^T (x^{**})^{K_i} - v(P) \leq U \frac{\ln K_i}{K_i}.$$

Since $\bar{x} \in F(AFP_{K_i})$ and $(x^*)^{K_i} \in F(AFP_{K_i})$, then $(x^{**})^{K_i} \in F(AFP_{K_i})$. By Lemma 5.1,

$$d((x^{**})^{K_i}, F(P)) \leq \sigma \max\left\{ \max_{t \in [0, 2\pi]} \sum_{j=1}^n a_j(t) (x^{**})_j^{K_i} - a_0(t), 0 \right\}.$$

Noticing that

$$\begin{aligned}
& \max_{t \in [0, 2\pi]} \sum_{j=1}^n a_j(t) (x^{**})_j^{K_i} - a_0(t) \\
&= \max_{t \in [0, 2\pi]} \sum_{j=1}^n (a_j(t) - \hat{a}_j^{K_i}(t)) (x^{**})_j^{K_i} - (a_0(t) - \hat{a}_0^{K_i}(t)) + \sum_{j=1}^n \hat{a}_j^{K_i}(t) (x^{**})_j^{K_i} - \hat{a}_0^{K_i}(t) \\
&\leq (2nR + 1)A \frac{\ln K_i}{K_i} + 2(nR + 1)A \frac{\ln K_i}{K_i} \\
&= (4nR + 3)A \frac{\ln K_i}{K_i},
\end{aligned}$$

so

$$d((x^{**})^{K_i}, F(P)) \leq \sigma(4nR + 3)A \frac{\ln K_i}{K_i}.$$

When $K_i \rightarrow \infty$, $d((x^{**})^{K_i}, F(P)) \rightarrow 0$. For any accumulation point x^{**} of $(x^{**})^{K_i}$, we have $c^T x^{**} = v(P)$, $\|x^{**}\| = 2R$, which is against to $\|S(P)\| \leq R$, so $\|S(FP_K)\| \leq 2R$, when K is large enough.

By Lemma 5.1, $\forall (x^*)^K \in S(FP_K)$,

$$d((x^*)^K, F(P)) \leq \sigma \max\left\{\max_{t \in [0, 2\pi]} \sum_{j=1}^n a_j(t) (x^*)_j^K - a_0(t), 0\right\}.$$

By the same discussion above, we could get

$$d((x^*)^K, F(P)) \leq \sigma(4nR + 3)A \frac{\ln K}{K},$$

so $(x^*)^K$ is a $\frac{\ln K}{K}$ -optimal solution of (P) when K is large enough.

(d) We will prove $((x^*)^K, (\bar{Y}^*)^K) \in S(TFP_K)$ with some $(\bar{Y}^*)^K \succeq 0$. Since the slater condition holds for Program (FP_K) , then, $v(TFP_K) \geq v(TFD_K) = v(FD_K) = v(FP_K)$, and

$$\begin{aligned}
0 &= \min \int_0^{2\pi} (\hat{a}_0^K(t) - \sum_{j=1}^n \hat{a}_j^K(t) (x^*)_j^K) \mu(dt) \\
&\quad s.t. \quad \mu \in \mathbf{P}([0, 2\pi]),
\end{aligned}$$

which means

$$\left\{ \begin{array}{l} \sum_{k=-K}^K (a_{0,k} - \sum_{j=1}^n a_{j,k} (x^*)_j^K) y_k < 0 \\ \begin{pmatrix} y_0 & y_1 & \cdots & y_K \\ y_{-1} & y_0 & \cdots & y_{K-1} \\ \vdots & \vdots & \ddots & \vdots \\ y_{-K} & y_{-K+1} & \cdots & y_0 \end{pmatrix} \succeq 0 \end{array} \right.$$

doesn't have solution. Then, there exists $Y^* \succeq 0$ such that

$$a_{0,k} - \sum_{j=1}^n a_{j,k}(x^*)_j^K = (\mathcal{C}_1^K(Y^*)_K)_k, k = -K, \dots, K.$$

So $(\bar{Y}^*)^K = \begin{pmatrix} (Y^*)_r^K & -(Y^*)_s^K \\ (Y^*)_s^K & (Y^*)_r^K \end{pmatrix} \succeq 0$, where

$$(Y^*)^K = (Y^*)_r^K + i(Y^*)_s^K, (Y^*)_r^K, (Y^*)_s^K \in \mathbb{R}^{(K+1) \times (K+1)},$$

and $((x^*)^K, (\bar{Y}^*)^K) \in F(TFP_K)$, which means $v(TFP_K) \leq v(FP_K)$ and $((x^*)^K, (\bar{Y}^*)^K) \in S(TFP_K)$. $\square$

Remark 5.1 In general, condition (b) in Theorem 5.1 does not hold for $a_j(t)$. If we replace $a_j(t)$ as

$$\bar{a}_j(t) = \begin{cases} a_j(2\pi - 2t), & 0 \leq t \leq \pi \\ a_j(2t - 2\pi), & \pi < t \leq 2\pi \end{cases},$$

then $\bar{a}_j(t)$ is Lipschitz continuous with $\bar{a}_j(0) = \bar{a}_j(2\pi)$ and could be considered as a periodic function with 2π period, while the feasible set of Program (P) is kept. Then, $\bar{a}_j(t)$ must be even function, which means

$$\frac{1}{2\pi} \int_0^{2\pi} \bar{a}_j(t) e^{ikt} dt$$

must be real, and $r_{j,k} = 0$. So Program (TFP_K) is equivalent to the following program denoted as Program $(RTFP_K)$:

$$\begin{aligned} (RTFP_K) \quad & \min \quad c^T x \\ & s.t. \quad \mathcal{C}_1^K Y + (E^K)^T x + p^K = 0 \\ & \quad Y \succeq 0, \end{aligned}$$

where $\mathcal{C}_1^K = \{\mathcal{C}_{1,j}^K\}_{j=0}^K$.

Corollary 5.1 If the conditions (a) and (b) in Theorem 5.1 are replaced by

- (a') $a_j(t)$ has q -th derivative $a_j^{(q)}(t)$ and $a_j^{(q)}(t)$ are Lipschitz continuous functions on $[0, 2\pi]$;
- (b') $a_j^{(q)}(0) = a_j^{(q)}(2\pi)$, $j = 0, \dots, n$;

respectively, x^* is a $\frac{\ln K}{K^{q+1}}$ -optimal solution of Program (P), when K is large enough, where $((x^*)^K, (Y^*)^K) \in S(TFP_K)$ with some $(Y^*)^K \succeq 0$.

Proof The proof procedure is similar to that of Theorem 5.1, except that we use Corollary III in page 22 of [8] instead of Corollary I. $\square$

As is evident from the foregoing discussion, in order to solve Program (TFP_K) or $(RTFP_K)$, both of which almost have the same parameters $a_{j,k}$, except the case $j = 0, k = 0$, the critical step in this algorithmic process is to efficiently compute the parameters $a_{j,k}$. Noting that

$$a_{j,k} = \frac{1}{2\pi} \int_0^{2\pi} a_j(t) e^{ikt} dt, \quad k = -K, \dots, K,$$

which is the k th coefficient of the Fourier transform of $a_j(t)$, we are only required to calculate $a_{j,k}$, for $k \geq 0$. In a general setting, we cannot accurately obtain $a_{j,k}$, and hence, we resort to using the

discrete Fourier transformations (DFT) to compute the necessary coefficients $a_{j,k}$ in a relatively quick fashion. The *fast Fourier transform (FFT)* is a well-known algorithm for computing discrete Fourier transforms, or their inverse [15]. A direct application of the DFT algorithm towards determining the coefficients $a_{j,k}$ results in a complexity of the order $O(K^2)$. But by exploiting the symmetry and periodicity of e^{ikt} , it is feasible to reduce this complexity to $O(K \ln K)$, where K is the truncation factor, which is a measure of the accuracy of the approximation and the data size involved in the problem.

Here, we present our algorithm.

Algorithm 5.1

Step 0: Given $\varepsilon > 0$, $N > 0$ large enough. Set

$$\bar{a}_j(t) = \begin{cases} a_j(2\pi - 2t), & 0 \leq t \leq \pi \\ a_j(2t - 2\pi), & \pi < t \leq 2\pi \end{cases}, \quad j = 0, \dots, n;$$

Step 1: Calculate $\bar{a}_{j,k}$, $k = 0, \dots, N, j = 0, \dots, n$ (by FFT), and get the real part $r_{j,k}$ and the imaginary part $s_{j,k}$ of them;

Step 2: Solving Program (TFP_K) or $(RTFP_K)$ to get the approximate optimal solution of Program (P) .

Remark 5.2 There are two variants of Algorithm 5.1 that have been used in our computational implementations, one that uses the FFT method and the other that does not, depending on the problem structure. Specifically, if the frequency of $a_j(t)$ is not too large, we can gainfully apply the FFT variant to generate good approximations of $a_{j,k}$, but the resulting $s_{j,k}$, i.e., the coefficients of the complex terms in the problem, may not be equal to zero. In this case, we would need to contend with solving the larger Program (TFP_K) as opposed to merely solving Program $(RTFP_K)$; Alternately, we can generate the values of $a_{j,k}$ by using other (perhaps more computationally intensive) schemes, but gain the advantage of solving the resulting relatively smaller Program $(RTFP_K)$.

6 Numerical Results

In this section, we demonstrate the efficiency of Algorithm 5.1 by solving five illustrative numerical examples taken from the literature [3, 17]. As the default index sets of these examples is not in the range $[0, 2\pi]$, we make some minor modifications to scale these programs into the required format. All of our computations are conducted on a Windows machine, equipped with a dual core 2.69GHz processor and 8GB RAM, using MATLAB R2011b as the computational engine. For solving the underlying semidefinite programs, we use SDPT4 and YALMIP software packages [10, 14]. These five examples are given below.

Example 1

$$\begin{aligned} \min \quad & \sum_{j=1}^n \frac{1}{j} x_j \\ \text{s.t.} \quad & \sum_{j=1}^n -\left(\frac{t}{2\pi}\right)^{j-1} x_j \leq -\tan\left(\frac{t}{2\pi}\right), \quad \forall t \in [0, 2\pi]. \end{aligned} \tag{18}$$

Example 2

$$\begin{aligned}
& \min \quad \sum_{j=1}^n x_j \\
& s.t. \quad \sum_{j=1}^n -\left(\frac{t}{2\pi} + 1\right)^{j-1} x_j \leq -\frac{2\pi}{\sqrt{4\pi^2 + t^2}}, \quad \forall t \in [0, 2\pi].
\end{aligned} \tag{19}$$

Example 3

$$\begin{aligned}
& \min \quad \sum_{j=1}^8 \frac{1}{j} x_j \\
& s.t. \quad \sum_{j=1}^8 -\left(\frac{t}{2\pi}\right)^{j-1} x_j \leq -\frac{2\pi}{4\pi - t}, \quad \forall t \in [0, 2\pi].
\end{aligned} \tag{20}$$

Example 4

$$\begin{aligned}
& \min \quad \sum_{j=1}^9 \frac{1}{j} x_j \\
& s.t. \quad \sum_{j=1}^9 -\left(\frac{t}{2\pi}\right)^{j-1} x_j \leq -\frac{4\pi^2}{4\pi^2 + t^2}, \quad \forall t \in [0, 2\pi].
\end{aligned} \tag{21}$$

Example 5

$$\begin{aligned}
& \min \quad \sum_{j=1}^{10} -(0.95)^{2j-1} x_j \\
& s.t. \quad \sum_{j=1}^{10} -2 \cos\left(\frac{(2j-1)t}{2}\right) x_j \leq 1, \quad \forall t \in [0, 2\pi].
\end{aligned} \tag{22}$$

We begin our computations by solving Example 1 and 2, corresponding to different values of $n \in \{5, 6, 7, 8\}$, where n denotes the dimension of x ; K is the truncation factor in Program (TFP_K) , which is also the number of Fourier terms presented in the program; ν_{Alg1} denotes the optimal value obtained by Algorithm 5.1; and the relative error is the absolute difference between the actual optimal value and the objective function value obtained by our algorithm. From [4, 17], the optimal values of Example 1, Example 2 for different dimensions are shown in Table 1. And by [3], the optimal values of Example 3, Example 4 and Example (5) are 0.69314815, 0.78549953, and -0.48354840 , respectively.

Table 1 Optimal values of Example 1 and Example 2, corresponding to different dimensions

n	5	6	7	8
Example 1	0.61740424	0.61608515	0.61572945	0.61565322
Example 2	1	1	1	1

We give the results of our algorithm in following tables. We use the FFT type of Algorithm 5.1 for Example 1, Example 2, Example 3, and Example 4, since the frequencies of all the functions $a_j(t)$ are not too large. For Example 5, we use Algorithm 5.1 without FFT. For comparing, we use the cutting plant method [4] to compute Example 1 and 2. For these two examples, we test 10 times and get the average results showed in Table 4 by using the cutting plant method.

Table 2 Numerical results by using Algorithm 5.1 on Example 1

n \ K	$\nu_{\text{Alg 5.1}}$			CPU Time (seconds)			Relative Error		
	8	16	32	8	16	32	8	16	32
5	0.6151	0.6164	0.6170	0.323	0.552	1.994	2.3×10^{-3}	1.0×10^{-3}	4.0×10^{-4}
6	0.6152	0.6158	0.6159	0.305	0.620	2.224	9.0×10^{-4}	3.0×10^{-4}	1.0×10^{-4}
7	0.6154	0.6156	0.6157	0.326	0.694	2.44	3.0×10^{-4}	1.0×10^{-4}	3.0×10^{-5}
8	0.6158	0.6156	0.6156	0.401	0.577	2.515	1.9×10^{-3}	2.0×10^{-4}	1.0×10^{-4}

Table 3 Numerical results by using Algorithm 5.1 on Example 2

n \ K	$\nu_{\text{Alg 5.1}}$			CPU Time (seconds)			Relative Error		
	8	16	32	8	16	32	8	16	32
5	1.00	1.00	1.00	0.388	0.774	3.397	2.7×10^{-6}	1.6×10^{-6}	3.0×10^{-7}
6	1.00	1.00	1.00	0.502	0.843	3.305	1.3×10^{-6}	8.3×10^{-9}	1.8×10^{-8}
7	1.00	1.00	1.00	0.397	0.775	3.151	2.0×10^{-7}	2.0×10^{-7}	4.9×10^{-8}
8	1.00	1.00	1.00	0.427	0.789	2.864	3.0×10^{-7}	1.0×10^{-7}	1.2×10^{-8}

Table 4 Numerical results by using the cutting plant method on Example 1 and Example 2

n	Example 1	CPU Time (seconds)	Relative Error
5	0.6165	0.437	9.0×10^{-4}
6	0.6154	0.520	6.0×10^{-4}
7	0.6155	0.598	5.5×10^{-4}
8	0.6154	0.588	7.0×10^{-4}
n	Example 2	CPU Time (seconds)	Relative Error
5	0.999	0.245	3.0×10^{-5}
6	1.000	0.334	8.0×10^{-6}
7	1.000	0.350	7.2×10^{-6}
8	1.000	0.421	4.3×10^{-6}

From the results recorded in Tables 2, 3, and 4, we find that our algorithm has better precision than the cutting plant method, although our times need a little more. But from the numerical tests, we found the results of the cutting plants are strongly depended on the start scattered point, which means that the optimal points are quite different when using the different start scattered point, while ours are more stable. Furthermore, from Tables 2 and 3, we infer that using a truncation factor $K \approx 2n$ yields an acceptable approximation to the original program (with objective function value within the prescribed tolerance limit), while yet expending only a reasonable computational effort.

The results for Example 3, Example 4 and Example 5 are shown in Table 5 respectively, where $K = 20$.

Table 5 Numerical results by using Algorithm 5.1 on Example 3, Example 4, and Example 5

Example	$\nu_{\text{Alg 5.1}}$	CPU Time (seconds)	Relative Error
3	0.6931	0.942	7.7×10^{-7}
4	0.7854	0.780	2.4×10^{-4}
5	-0.4835	0.411	1.2×10^{-9}

We could see that our algorithm is efficient and effective.

7 Conclusions

In this paper, we present a framework to determine solutions for a class of linear semi-infinite programs based on the theory of trigonometric moments. Beginning with the dual program to the linear semi-infinite program, we first construct an approximation to the dual program using Fourier transformations, and subsequently transform the truncated Fourier dual program into an approximate semidefinite program. Recognizing that the derived semidefinite program is not a standard optimization problem due to the presence of complex variables, we cleverly exploit the properties of the underlying matrices to derive an equivalent real-valued semidefinite program. Moreover, we also discuss various relationships between the original semi-infinite program and the derived semidefinite program, and prove that the optimal solution of the truncated semidefinite program does indeed be the approximate optimal solution of the original semi-infinite program. An algorithm based on Fast Fourier Transforms is presented, and our illustrative computational results serve to showcase the computational efficiency of the algorithm.

Acknowledgements: Y. Xu was supported in part by National Natural Science Foundation of China No. 11501100, 11571178, 11671082 and 11871149. J. Desai was supported in part by the Ministry of Education (Singapore) Academic Research Fund Tier 1 Grant No. M4011083. X. Yan was supported in part by the STIP of Higher Education Institutions in Shanxi No. 201802103, and National Natural Science Foundation of China No. 11901424.

References

1. Akhiezer, N.I.: The classical moment problem. Hafner, New York (1965)
2. Akhiezer, N.I., Krein, M.G.: Some questions in the theory of moments. American Mathematical Society Translations Vol. 2 (1962)

3. Betrò, B.: An accelerated central cutting plane algorithm for linear semi-infinite programming. *Math. Program.*, 101, 479-495 (2004)
4. Goberna, M.A., López, M.A.: *Linear semi-infinite optimization*. Chichester: John Wiley & Sons (1998)
5. Hettich, R., Kortanek, K.O.: *Semi-infinite programming: theory, methods, and applications*. *SIAM Rev.*, 35:380-429 (1993)
6. Lasserre, J.B.: *Moments, positive polynomials and their applications*. Imperial College Press Optimization Series, vol. 1, Imperial College Press, London (2010)
7. Lasserre, J.B.: Global optimization with polynomials and the problem of moments. *SIAM Journal on Optimization*, 11(3), 796-817 (2001)
8. Jackson, D.: *The theory of approximation*. American Mathematical Society, New York (1930)
9. Ling, C., Ni, Q., Qi, L., Wu, S.Y.: A new smoothing Newton-type algorithm for semi-infinite programming. *J. Glob. Optim.*, 47, 133-159 (2010)
10. Löfberg, J.: YALMIP : A Toolbox for Modeling and Optimization in MATLAB. In *Proceedings of the CACSD Conference*, Taipei, Taiwan (2004)
11. Luo, Z-Q, Tseng, P.: Perturbation analysis of a condition number for linear systems, *SIAM J. Matrix Anal. Appl.* Vol. 15, No. 2, 636-660, (1994)
12. Reemtsen, R., Rückmann, J-J, ,editors: *Semi-infinite programming*. Boston: Kluwer Academic (1998)
13. Stein, O., Still, G.: Solving semi-infinite optimization problems with interior point techniques. *SIAM J. Control Optim.*, 42, 769-788 (2003)
14. Tutuncu, R.H., Toh, K.C., Todd, M.J.: Solving semidefinite-quadratic-linear programs using SDPT3. *Math. Prog.*, 95, 189-217 (2003)
15. van Loan, C.: *Computational Frameworks for the Fast Fourier Transform*. Philadelphia: SIAM (1992)
16. Wolkowicz, H., Saigal, R., Vandenberghe, L.: *Handbook of Semidefinite Programming*. Kluwer (2000)
17. Xu, Y., Sun, W.Y., Qi, L.: On solving a class of linear semi-infinite programming by SDP method. *Optimization*, 64(3), 603-616 (2015)